

Week 2 Day 2 Hands-on Lab: Dockerfile, Image Build, Disk Persistence

목적

이 문서는 Day 2의 전체 실습 범위를 하나의 긴 실행 흐름으로 묶는다. 교시별 lesson은 개념과 수업 진행을 설명하고, 이 lab guide는 학생이 실제로 따라 치고 기록할 명령, 기대 결과, 실패 drill, cleanup 기준을 제공한다.

Day 2 실습은 네 가지 질문을 끝까지 확인한다.

- 어떤 파일이 build context로 들어가는가?
- Dockerfile instruction은 image filesystem을 어떻게 바꾸는가?
- build한 image가 실제로 HTTP 서비스를 제공하는가?
- container 삭제와 disk 데이터 보존은 어떻게 다르게 동작하는가?

실습 시간 배분

Phase	범위	권장 시간	핵심 evidence
A	실습 앱 구조 확인	15분	pwd, ls -la, Dockerfile
B	Dockerfile 읽기	20분	instruction 역할표
C	image build	25분	build context, COPY step, tag
D	container run/check	25분	docker ps, curl -I, body text
E	build troubleshooting	25분	failure note, history, inspect
F	bind mount disk 연동	35분	v1/v2 응답, Mounts
G	named volume persistence	25분	write/read/volume cleanup
H	README evidence 정리	20분	build/run/disk/troubleshoot

전체 실행은 2일차 안에서 교시별로 나누어 진행한다. 빠른 학생은 failure drill과 개인 Week 1 앱 적용으로 확장한다.

공통 준비

repository root에서 시작한다.

```
pwd
docker version
docker ps
```

정상 기준:

- pwd 가 수업 repository root를 가리킨다.
- docker version 에서 Client와 Server가 모두 보인다.
- 이전 실습 container가 남아 있지 않다.

정리 명령:

```
docker rm -f paperclip-day2-static paperclip-day2-disk 2>/dev/null || true
docker volume rm paperclip-day2-data 2>/dev/null || true
```

위 명령은 교육 중 복구용이다. 실제 운영 데이터에는 rm -f 와 volume rm 을 신중하게 사용한다.

Phase A: 실습 앱 구조 확인

```
cd week2/day2/labs/static-site
pwd
ls -la
sed -n '1,120p' Dockerfile
sed -n '1,120p' .dockerignore
```

기대 파일:

```
Dockerfile
.dockerignore
index.html
styles.css
README.md
```

기록할 것:

- build context directory:
- Dockerfile path:
- COPY 대상 파일:
- .dockerignore 가 제외하는 파일:

Phase B: Dockerfile instruction 분석

Dockerfile:

```
FROM nginx:1.27-alpine

WORKDIR /usr/share/nginx/html

COPY index.html styles.css ./

EXPOSE 80
```

분석표:

Instruction	build/runtime	filesystem 변화	운영 의미
FROM nginx:1.27-alpine	build	base layer 선택	재현성과 보안의 시작점
WORKDIR /usr/share/nginx/html	build metadata	기존 path 변경	이후 COPY 대상 기준
COPY index.html styles.css ./	build	image에 파일 추가	앱 소스 포함
EXPOSE 80	metadata	port metadata	container 내부 port 문서화

질문:

- EXPOSE 80 만 있으면 host browser에서 바로 접속되는가?
- COPY . ./ 가 아니라 파일명을 지정한 이유는 무엇인가?
- nginx:latest 대신 nginx:1.27-alpine 을 쓴 이유는 무엇인가?

Phase C: image build

```
docker build -t paperclip-static-site:day2 .
```

Linux 사전 테스트 핵심 출력:

```
#3 [internal] load .dockerignore
#4 [internal] load build context
#4 transferring context: 2.42kB
#6 [2/3] WORKDIR /usr/share/nginx/html
#7 [3/3] COPY index.html styles.css ./
#8 naming to docker.io/library/paperclip-static-site:day2 done
```

기록할 것:

- build context size:
- COPY step number:
- image tag:
- build success/failure:

Phase C-2: cache 재빌드 확인

같은 명령을 한 번 더 실행한다.

```
docker build -t paperclip-static-site:day2 .
```

확인할 것:

- 변경이 없으면 일부 step이 cache로 빠르게 처리될 수 있다.
- source를 바꾸면 COPY 이후 step이 다시 처리될 수 있다.

강제 재빌드:

```
docker build --no-cache -t paperclip-static-site:day2-nocache .
```

수치화 측정:

```
/usr/bin/time -f 'elapsed=%e sec' docker build -t paperclip-static-site:metrics .
/usr/bin/time -f 'elapsed=%e sec' docker build --no-cache -t paperclip-static-site:metrics-nocache .
docker history paperclip-static-site:metrics
docker images paperclip-static-site
```

Linux 사전 테스트 측정값:

```
cache build: elapsed=1.54 sec, WORKDIR/COPY step CACHED
no-cache build: about 1.50 sec on this tiny example
image size: 48.2MB
COPY index.html styles.css ./ layer: 2.34kB
source layer ratio: about 0.0049% of the full image
```

주의: 이 예제는 HTML/CSS 두 파일만 복사하므로 build time 차이가 작다. 여기서 중요한 값은 COPY layer가 전체 image보다 훨씬 작다는 점이다. 소스만 바뀌면 base image와 nginx 설치 layer는 재사용되고, 학생이 바꾼 소스 layer만 새로 만들어진다.

기록할 것:

- cache build와 no-cache build의 체감 차이:
- CACHED 가 표시된 step:
- 전체 image size:

- COPY layer size:
- 변경 layer가 전체 image에서 차지하는 비율:
- 언제 --no-cache 가 필요한가:

Phase D: container run/check

```
docker run -d --name paperclip-day2-static -p 18082:80 paperclip-static-site:day2
docker ps --filter name=paperclip-day2-static
curl -I http://localhost:18082
curl -s http://localhost:18082 | grep "Dockerfile로 만든 표준 실습 앱"
docker logs paperclip-day2-static
docker exec paperclip-day2-static ls -l /usr/share/nginx/html
```

Linux 사전 테스트 핵심 결과:

```
0.0.0.0:18082->80/tcp
HTTP/1.1 200 OK
Server: nginx/1.27.5
<h1>Dockerfile로 만든 표준 실습 앱</h1>
index.html
styles.css
```

기록할 것:

- container ID:
- port binding:
- HTTP status:
- expected text:
- internal file list:

정리:

```
docker stop paperclip-day2-static
docker rm paperclip-day2-static
docker ps --filter name=paperclip-day2-static
```

정상 기준: 마지막 명령은 헤더만 출력한다.

Phase E: failure drill

Drill 1: build context 밖 파일 COPY

Dockerfile을 직접 망가뜨리지 말고, 아래를 별도 파일로 만들어 관찰한다.

```
cp Dockerfile Dockerfile.bad-copy
```

Dockerfile.bad-copy 에 다음 줄을 임시로 넣는 상황을 가정한다.

```
COPY ../secret.txt ./
```

예상 실패:

- build context 밖 파일은 복사할 수 없다.
- 조치는 context boundary를 바꾸는 것이 아니라 필요한 파일을 context 안에 안전하게 두는 것이다.

Drill 2: port 충돌

이미 `paperclip-day2-static` 이 떠 있는 상태에서 같은 port로 다시 실행하면 실패할 수 있다.

```
docker run -d --name paperclip-day2-static-2 -p 18082:80 paperclip-static-site:day2
```

확인:

```
docker ps
docker logs paperclip-day2-static-2
```

조치:

```
docker rm -f paperclip-day2-static-2
```

Drill 3: cache 의심

source를 바꿨는데 결과가 안 바뀐 것처럼 보이면 다음 순서로 확인한다.

```
curl -s http://localhost:18082
docker build --no-cache -t paperclip-static-site:day2 .
docker rm -f paperclip-day2-static
docker run -d --name paperclip-day2-static -p 18082:80 paperclip-static-site:day2
curl -s http://localhost:18082
```

주의: browser cache와 Docker build cache를 혼동하지 않는다. `curl` 은 browser cache 영향을 줄이는 확인 도구다.

Phase F: bind mount disk 연동

repository root로 돌아간다.

```
cd /mnt/d/paperclip
docker run -d \
  --name paperclip-day2-disk \
  -p 18083:80 \
  -v "$PWD/week2/day2/labs/disk-mount/html:/usr/share/nginx/html:ro" \
  nginx:1.27-alpine

curl -s http://localhost:18083
docker inspect paperclip-day2-disk
```

v1 정상 출력:

```
<h1>Disk mount lab - host file v1</h1>
```

host 파일을 수정한다.

```
<h1>Disk mount lab - host file v2</h1>
<p>host disk 파일을 수정하자 container 응답이 바뀐다.</p>
```

다시 확인한다.

```
curl -s http://localhost:18083
```

v2 정상 출력:

```
<h1>Disk mount lab - host file v2</h1>
```

inspect에서 확인할 mount:

```
"Type": "bind"
"Source": ".../week2/day2/labs/disk-mount/html"
"Destination": "/usr/share/nginx/html"
"Mode": "ro"
"RW": false
```

정리:

```
docker stop paperclip-day2-disk
docker rm paperclip-day2-disk
```

Phase G: named volume persistence

```
docker volume create paperclip-day2-data

docker run --rm \
  -v paperclip-day2-data:/data \
  alpine:3.20 \
  sh -c "echo volume-note-v1 > /data/note.txt && cat /data/note.txt"
```

정상 출력:

```
paperclip-day2-data
volume-note-v1
```

새 container에서 다시 읽는다.

```
docker run --rm \
  -v paperclip-day2-data:/data \
  alpine:3.20 \
  cat /data/note.txt
```

정상 출력:

```
volume-note-v1
```

inspect:

```
docker volume inspect paperclip-day2-data
```

정리:

```
docker volume rm paperclip-day2-data
docker volume ls --filter name=paperclip-day2-data
```

정상 기준: 마지막 명령은 헤더만 출력한다.

Phase H: README evidence 작성

아래 내용을 개인 repository README 또는 실습 note에 작성한다.

Docker Day 2 Evidence

Build

- Directory:
- Dockerfile:
- Build command:
- Build context size:
- Image tag:

Run and Check

- Run command:
- Container name:
- Port:
- HTTP status:
- Expected body text:
- Log evidence:
- Internal file evidence:

Build Troubleshooting

- Failure drill:
- Evidence:
- Fix:
- Recheck:

Disk Mount

- Bind source:
- Bind destination:
- Mode:
- v1 response:
- v2 response:

Named Volume

- Volume name:
- Write result:
- Read-after-container-removal result:
- Cleanup:

최종 cleanup audit

```
docker ps --filter name=paperclip-day2-static
docker ps --filter name=paperclip-day2-disk
docker volume ls --filter name=paperclip-day2-data
```

세 명령 모두 헤더만 출력하면 Day 2 실습 resource가 정리된 상태다.

평가 기준

기준	2점 evidence
Build	context size, COPY step, image tag를 기록했다.

Run	port binding, HTTP 200, expected body text를 확인했다.
Inspect	image/container/volume 중 하나 이상을 inspect했다.
Failure	실패 drill 하나를 증상, 원인, 조치, 재확인으로 기록했다.
Disk	bind mount v1/v2와 named volume persistence를 모두 확인했다.
Cleanup	container와 volume 잔여 상태를 확인했다.